KAFKA

Exercice 1 : Mise en place de Kafka et d'un producteur simple

1. Créez un topic Kafka nommé `weather_stream` .
2. Envoyez un message statique, par exemple :

```
{"msg": "Hello Kafka"}
```

Exercice 2 : Écriture d'un consommateur Kafka

1. Créez un script Python consommateur qui lit les messages depuis un topic Kafka défini en variable au début du script.
2. Affichez les messages reçus en temps réel.

Exercice 3 : Streaming de données météo en direct

1. Écrivez un producteur `current_weather` qui interroge l'API Open-Meteo toutes les 30 secondes pour chacune des villes suivantes :

city	country	latitude	longitude
Paris	France	48.8566	2.3522
Seville	Spain	37.3891	-5.9845
Dubai	United Arab Emirates	25.2048	55.2708
Reykjavik	Iceland	64.1466	-21.9426
Wellington	New Zealand	-41.2865	174.7762

   URL de l'API :

```
https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m,wind_speed_10m,weather_code&wind_speed_unit=ms
```

2. Envoyez la réponse de l'API dans le topic Kafka `weather_stream` , en y ajoutant les champs `city` et `country` .

Exercice 4 : Transformation des données et détection d'alertes

1. Traitez le flux `weather_stream` en temps réel avec **Spark**.
2. Produisez un topic Kafka `weather_transformed` contenant :
   - **Alertes de vent** :
     - Vent faible (< 10 m/s) → `level_0`
     - Vent modéré (10–20 m/s) → `level_1`
     - Vent fort (> 20 m/s) → `level_2`
   - **Alertes de chaleur** :
     - Température normale (< 25°C) → `level_0`
     - Chaleur modérée (25–35°C) → `level_1`
     - Canicule (> 35°C) → `level_2`

3. Ajoutez les colonnes suivantes :
    - `event_time` (timestamp) : l'heure d'écriture du message dans Kafka (colonne `timestamp` de la source Kafka)
    - `temperature` et `windspeed` transformés si nécessaire
    - `city` , `country` et `weather_code`
    - `wind_alert_level` et `heat_alert_level`

Exercice 5 : Agrégats en temps réel avec Spark

Goal : Calculer des agrégats en temps réel sur une fenêtre glissante.

1. Lisez le flux `weather_transformed` avec Spark.
2. Implémentez un **sliding window** de **1 minute** qui avance toutes les **30 secondes**, sur `event_time` , avec un watermark de 1 minute.
3. Pour chaque fenêtre et chaque ville (`country` , `city`), calculez :
    - le nombre d'alertes vent `level_1` et `level_2`
    - le nombre d'alertes chaleur `level_1` et `level_2`
    - le nombre total d'alertes
    - la moyenne, le min et le max de la température
4. Affichez le résultat dans la console en mode `update` .

Exercice 6 : Stockage dans HDFS organisé

1. Écrivez un consommateur Kafka qui lit `weather_transformed` en continu.
2. Sauvegardez chaque message reçu dans **HDFS**, dans le fichier de sa ville :
    - Structure : `/hdfs-data/{country}/{city}/alerts.json`
    - Une ligne JSON par message, ajoutée à la fin du fichier : le fichier n'est jamais réécrit.
    - Tous les messages sont gardés, `level_0` compris : l'exercice 7 trace l'évolution complète de la température et du vent.

Exercice 7 : Visualisation et agrégation des logs météo

Consommer les logs HDFS et Implémentez les visualisations suivantes :

- Évolution de la température au fil du temps
- Évolution de la vitesse du vent
- Nombre d'alertes vent et chaleur par niveau
- Code météo le plus fréquent par pays